

Module 09 — Amortized Analysis

Sources: CLRS 4e ch. 16 (Amortized Analysis) · Skiena 3e §3.1.1 (dynamic arrays), §15.1 (dictionaries catalog note), Problem 3-5

Big Idea

Worst-case-per-operation analysis is sometimes **correct but not tight**, and the gap can be an entire factor of n . A stack `MULTIPOP` can pop a thousand items — but only if a thousand items were pushed first. A dynamic array doubling costs $\Theta(n)$ — but only once every n insertions. If you bound each operation by its own worst case and multiply by n , you get $O(n^2)$ for a sequence that actually costs $O(n)$.

Amortized analysis is the technique for bounding the total cost of a sequence of operations, then dividing. The result is a per-operation cost that is *not* an average over random inputs — it is a **worst-case guarantee about the sequence**. CLRS is emphatic on this: *"Amortized analysis differs from average-case analysis in that probability is not involved. An amortized analysis guarantees the average performance of each operation in the worst case."*

The chapter's opening analogy: Buff's Gym charges \$60/month plus \$3 per visit. Visit every day in November and you pay $60 + 3 \times 30 = \$150$ over 30 days — an average of \$5 per day. You have **amortized** the flat monthly fee over the days, spreading it at \$2/day. Nothing random happened; you just moved a lump-sum charge onto the operations that made it necessary.

There are three techniques, and they are three *views of the same accounting*, not three different results:

METHOD	YOU COMPUTE	ALL OPERATIONS SAME COST?	BEST WHEN
Aggregate	a bound $T(n)$ on the <i>total</i> , then $T(n)/n$	yes	one operation type, or a clean counting argument exists
Accounting	a per-operation charge; surplus is stored as credit on specific objects	no	you can point at <i>which object</i> prepays for the expensive step
Potential	a per-operation charge; surplus is potential energy Φ of the whole structure	no	credit isn't naturally per-object; the state itself is the right bookkeeping

Remember months later: *expensive operations are paid for by the cheap operations that made them possible. Find the quantity that the cheap operations build up and the expensive operation destroys — that quantity is your potential function.*

And one warning CLRS repeats: **the charges are for analysis only. They must not appear in the code.** There is no `x.credit` field.

What You Should Be Able To Do After This Chapter

- Explain why the naive bound on n stack operations is $O(n^2)$ and why the truth is $O(n)$.
- Run all three methods on both running examples (`MULTIPOP` stack, binary counter) from memory.
- State the accounting method's constraint $\sum \hat{c}_i \geq \sum c_i$ and explain why credit must never go negative.
- State the potential method's definition $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$, derive the telescoping sum, and say what condition makes $\sum \hat{c}_i$ an upper bound on $\sum c_i$.
- Derive the dynamic-table potential $\Phi(T) = 2(\text{num} - \text{size}/2)$ from scratch — i.e. *design* it rather than recall it.
- Explain why "halve when less than half full" is $\Theta(n)$ amortized and why "halve when less than **quarter** full" is $O(1)$.
- Recognize the pattern in a new problem and pick a potential function.
- Distinguish amortized, worst-case, and average-case, and rank them (Skiena's ordering).

Part 1 — What Amortized Analysis Is

Unified Understanding

In an amortized analysis you average the time required to perform a **sequence** of data-structure operations over all the operations performed. The conclusion has the form: *any sequence of n operations costs $O(f(n))$ in total, so the amortized cost per operation is $O(f(n)/n)$ — even though one operation within the sequence may cost much more.*

The three notions of "cost per operation", ranked — Skiena states the ordering precisely (§15.1):

"A data structure realizing an amortized complexity of $O(f(n))$ is less desirable than one whose worst-case complexity is $O(f(n))$ (since a very bad operation might still occur) but better than one with an average-case complexity $O(f(n))$, since the amortized bound will achieve this average on any input."

GUARANTEE	SAYS	FAILS WHEN
Worst-case $O(f(n))$ per op	every operation is fast	never — strongest
Amortized $O(f(n))$ per op	any <i>sequence</i> of m ops costs $O(m \cdot f(n))$; no probability ; holds for every input	a single operation can still stall (bad for real-time / latency SLOs)
Average-case $O(f(n))$ per op	fast <i>in expectation</i> over a distribution of inputs	the adversary picks the input

Why the middle row is a real guarantee and the bottom row often isn't: the amortized bound holds on **any input**, including adversarial ones. The average-case bound assumes a distribution that reality may not supply. This is the same distinction as randomized-algorithm-vs-probabilistic-analysis in M04 — and the same reason a randomized quicksort's expected $O(n \lg n)$ is worth more than a deterministic quicksort's average-case $O(n \lg n)$.

Where amortized bounds hurt. Skiena, on dynamic arrays: *"The primary thing lost in using dynamic arrays is the guarantee that each insertion takes constant time in the worst case... What we get instead is a promise that the n th element insertion will be completed quickly enough that the total effort expended so far will still be $O(n)$."* If you are writing a game loop, an audio callback, or a system with a tail-latency budget, that occasional $\Theta(n)$ stall is exactly what you cannot afford — which is why real-time systems preallocate and

why some libraries offer *incremental* rehashing that spreads the work out.

Running example 1 — a stack with MULTIPOP

Ordinary `PUSH` and `POP` cost 1 each. Add:

```
MULTIPOP(S, k)
1  while not STACK-EMPTY(S) and k > 0
2      POP(S)
3      k = k - 1
```

→ **C++ implementation:** [A1 MULTIPOP](#)

The actual cost of `MULTIPOP(S, k)` on a stack of `s` objects is $\min\{s, k\}$.

The naive bound. A `MULTIPOP` can cost $O(n)$ since the stack holds at most `n` items. There are at most `n` operations. Therefore $O(n^2)$. **This is correct and useless.**

Why it's loose: *an object cannot be popped unless it was first pushed.* The number of `POP` calls — including those inside `MULTIPOP` — is at most the number of `PUSH` calls, which is at most `n`. So `n` operations cost $O(n)$ total, and the amortized cost of each of the three operations is $O(1)$.

Verified: 200 000 random push-heavy operations performed 340 269 elementary pushes/pops in total, comfortably under the $2n = 400\,000$ bound — while the most expensive single `MULTIPOP` popped 156 items.

Running example 2 — incrementing a `k`-bit binary counter

`A[0..k-1]` holds bits, `A[0]` least significant, so the counter's value is $x = \sum A[i] \cdot 2^i$.

```
INCREMENT(A, k)
1  i = 0
2  while i < k and A[i] == 1
3      A[i] = 0
4      i = i + 1
5  if i < k
6      A[i] = 1
```

→ C++ implementation: [A2 INCREMENT](#)

The cost is the number of bits flipped.

The naive bound. One `INCREMENT` flips up to k bits (when the counter is all 1s), so n increments cost $O(nk)$. Correct, not tight.

Why it's loose: *not all bits flip on every call.* Over n increments starting from 0:

BIT	FLIPS HOW OFTEN	TOTAL FLIPS IN n INCREMENTS
<code>A[0]</code>	every time	n
<code>A[1]</code>	every other time	$\lfloor n/2 \rfloor$
<code>A[2]</code>	every fourth time	$\lfloor n/4 \rfloor$
<code>A[i]</code>	every 2^i -th time	$\lfloor n/2^i \rfloor$

$$\text{total flips} = \sum_{i=0}^{k-1} \lfloor n/2^i \rfloor < n \cdot \sum_{i=0}^{\infty} 1/2^i = 2n$$

So n increments cost $O(n)$ and the amortized cost per `INCREMENT` is $O(1)$ — in fact **at most 2 bit flips per increment**.

Verified: 200 000 `INCREMENT`s on a 40-bit counter flipped exactly **399 994** bits — within 6 of the $2n = 400\,000$ bound, with the worst single increment flipping 18 bits.

Note the structural similarity CLRS points out: a single `MULTIPOP` might pop many objects, but not every call pops many; a single `INCREMENT` might flip all k bits, but not every call flips many. That similarity is the recognition signal for amortized analysis.

Part 2 — The Three Methods

Method 1: Aggregate analysis

Procedure. Show that for all n , a sequence of n operations takes $T(n)$ worst-case time **in total**. Then the amortized cost per operation is $T(n)/n$, assigned uniformly to every operation regardless of type.

Strengths: shortest when a clean global counting argument exists ("every pop is matched to a push"; "bit i flips $n/2^i$ times").

Weakness: all operations get the *same* amortized cost, which can be a bad fit when the operation types genuinely differ (e.g. you'd like `INSERT` to be $O(\lg n)$ amortized and `EXTRACT-MIN` to be $O(1)$ amortized — aggregate analysis can't say that).

Method 2: The accounting method

Procedure. Assign each operation type an **amortized cost** $\hat{c}_i$ (a charge), possibly different from its **actual cost** c_i . When $\hat{c}_i > c_i$, the surplus is stored as **credit on a specific object** in the data structure. When $\hat{c}_i < c_i$, the shortfall is paid from credit already on the objects involved.

The constraint you must verify:

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i \quad \text{for every sequence of } n \text{ operations} \quad (16.1)$$

Equivalently: **the total credit must never go negative.** Total credit is $\sum \hat{c}_i - \sum c_i$, so if it ever dipped below zero, the amortized total would stop being an upper bound at that instant — and an upper bound that holds only sometimes is no bound at all.

Stack. Actual costs: `PUSH` 1, `POP` 1, `MULTIPOP` $\min\{s, k\}$. Charge:

OPERATION	ACTUAL	AMORTIZED (CHARGE)
<code>PUSH</code>	1	2
<code>POP</code>	1	0
<code>MULTIPOP</code>	$\min\{s, k\}$	0

Think of plates in a cafeteria stack. Pushing a plate costs \$2: \$1 pays the actual push, and **\$1 is placed on the plate**. Every plate on the stack carries \$1. Popping a plate costs \$1, paid by taking the dollar off that plate — so `POP` is free. `MULTIPOP` is just repeated free pops. The number of plates is never negative, so the credit is never negative. Total amortized cost $2n = O(n)$ bounds the total actual cost. ■

Binary counter. Charge **\$2 to set a 0-bit to 1**: \$1 does the actual flip, \$1 sits on the bit as credit for when it is later reset to 0. Resetting is then free (paid by the dollar on the bit). Each `INCREMENT` sets **at most one bit to 1** (line 6), so its amortized cost is at most \$2. The number of 1-bits is never negative, so credit stays nonnegative. Total $O(n)$. ■

The insight to steal: in both cases, the credit sits on *the thing that will later be expensive to destroy*. That is how to invent a charging scheme — find the object whose destruction is the expensive part, and prepay it at creation.

Method 3: The potential method

Procedure. Instead of per-object credit, define a **potential function** Φ mapping each state D_i of the data structure to a real number. Starting from D_0 , operation i has actual cost c_i and transforms D_{i-1} into D_i . Define:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

(16.2)

The amortized cost is the actual cost plus the change in potential. Summing telescopes:

$$\sum \hat{c}_i = \sum (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

(16.3)

The condition. If $\Phi(D_n) \geq \Phi(D_0)$, the total amortized cost bounds the total actual cost. Since you usually don't know n in advance, require $\Phi(D_i) \geq \Phi(D_0)$ for **all** i . Simplest of all: **set $\Phi(D_0) = 0$ and prove $\Phi(D_i) \geq 0$ for all i .**

Reading the sign. $\Delta\Phi_i > 0$ means the operation was **overcharged** and the structure stored energy; $\Delta\Phi_i < 0$ means the operation was **undercharged** and the release of stored energy paid for it. Note that an amortized cost can legitimately be **negative** (a deletion that empties a lot of potential) — that's fine, it just means that operation is subsidizing others.

Stack. $\Phi(D) =$ the number of objects on the stack. $\Phi(D_0) = 0$, $\Phi \geq 0$ always. Then:

OPERATION	c_i	$\Delta\Phi_i$	$\hat{c}_i$
PUSH	1	$(s+1) - s = +1$	2
POP	1	-1	0
MULTIPOP, popping $k' = \min\{s, k\}$	k'	$-k'$	0

All $O(1)$, so n operations cost $O(n)$. ■

Binary counter. $\Phi(D_i) = b_i$, the number of 1-bits after operation i . Suppose the i -th **INCREMENT** resets t_i bits to 0. Then $c_i \leq t_i + 1$ (reset t_i bits, set at most one). For the potential: if $b_i = 0$ the operation reset all k bits so $b_{i-1} = t_i = k$; otherwise $b_i = b_{i-1} - t_i + 1$. **Either way $b_i \leq b_{i-1} - t_i + 1$** , so

$$\begin{aligned}\Delta\Phi_i &\leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i \\ \hat{c}_i = c_i + \Delta\Phi_i &\leq (t_i + 1) + (1 - t_i) = 2\end{aligned}$$

■ Notice the t_i cancels — that cancellation is the amortized argument, and finding a Φ that makes the expensive term cancel is the whole game.

The bonus the potential method gives you for free: a counter that doesn't start at zero. Rewrite (16.3) as $\sum c_i = \sum \hat{c}_i - \Phi(D_n) + \Phi(D_0)$. With $\Phi(D_0) = b_0$, $\Phi(D_n) = b_n$, and $\hat{c}_i \leq 2$:

$$\sum c_i \leq 2n - b_n + b_0$$

Since $b_0 \leq k$, as long as $k = O(n)$ the total actual cost is $O(n)$ **no matter what value the counter started at**. Aggregate and accounting analyses don't hand you this; the potential method does, because $\Phi(D_0)$ is an explicit term in the identity.

Choosing a potential function — the practical recipe

1. **Identify the expensive operation** and what makes it expensive (copying m items, popping s items, flipping t bits).
2. **Identify the quantity the cheap operations build up** that the expensive operation consumes. Usually it is literally "how far the structure has drifted from the state it's in right after the expensive operation".
3. **Set $\Phi = 0$ at the state right after the expensive operation**, and scale Φ so that by the time the expensive operation is triggered again, Φ has grown to (at least) its cost.
4. **Verify $\Phi \geq 0$ always** and $\Phi(D_0) = 0$.
5. **Compute $\hat{c}_i = c_i + \Delta\Phi_i$ for every case**, including the boundary cases where the structure crosses a threshold.

Step 3 is the one people skip, and it is the one that actually determines the constant. The dynamic-table derivation below is step 3 done explicitly.

Part 3 — Dynamic Tables (CLRS 16.4, Skiena 3.1.1)

The problem

You don't know in advance how many items a table will hold. Allocate too little and you must reallocate and copy; allow too many deletions and you waste space. `TABLE-INSERT` adds one item, `TABLE-DELETE` removes one; the underlying structure could be a stack, a heap, a hash table, anything. The **load factor** $\alpha(T) = \text{num} / \text{size}$ (defined as 1 for an empty table with `size = 0`) measures the waste: **if α is bounded below by a positive constant, unused space is never more than a constant fraction of the total.**

Two properties we want simultaneously:

- the load factor is bounded below by a positive constant (and above by 1);
- the amortized cost of a table operation is bounded above by a constant.

16.4.1 — Expansion only

```
TABLE-INSERT(T, x)
1  if T.size == 0
2      allocate T.table with 1 slot
3      T.size = 1
4  if T.num == T.size
5      allocate new-table with 2 * T.size slots
6      insert all items in T.table into new-table          // the
expensive part
7      free T.table
8      T.table = new-table
9      T.size = 2 * T.size
10 insert x into T.table
11 T.num = T.num + 1
```

→ **C++ implementation:** [A3 TABLE-INSERT](#) (and `TABLE-DELETE`)

Actual cost, counting elementary insertions:

```
ci = i    if i - 1 is an exact power of 2    (expansion: copy i-1
items, insert 1)
        1    otherwise
```

Aggregate analysis:

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j < n + 2n = 3n$$

so the amortized cost is at most **3**. (Skiena derives the same $2n$ copying bound from the doubling side: recopying happens after the 1st, 2nd, 4th, ..., n -th insertions with 2^{i-1} moves at the i -th doubling, giving $M \leq 2n$, so "**each of the n elements move only two times on average**".)

Accounting method — why the constant is 3. Each item pays for **three** elementary insertions:

1. inserting **itself** into the current table,
2. moving **itself** the next time the table expands,
3. moving **some other item** that was already in the table at that expansion.

Concretely: right after an expansion the table has size m and holds $m/2$ items, with no credit. Each `TABLE-INSERT` charges \$3 — \$1 for the immediate insertion, \$1 on the new item, \$1 on one of the $m/2$ old items. The table won't fill for another $m/2 - 1$ insertions, and by the time it holds m items **every item carries \$1**, exactly enough to move all m of them.

Potential method — deriving Φ rather than recalling it. Follow the recipe:

- Set $\Phi = 0$ at the post-expansion state, i.e. when $T.\text{num} = T.\text{size}/2$.
- From there, $T.\text{size}/2$ more insertions fill the table; the expansion then costs $T.\text{size}$.
- So Φ must climb from 0 to $T.\text{size}$ over $T.\text{size}/2$ insertions, i.e. **+2 per insertion**.

That forces:

$$\Phi(T) = 2 \cdot (T.\text{num} - T.\text{size}/2) \\ (16.4)$$

Check: $\Phi = 0$ when $\text{num} = \text{size}/2$ ✓; $\Phi = \text{size}$ when the table is full ✓; $\Phi \geq 0$ because the table (under insert-only doubling) is always at least half full ✓; $\Phi(T_0) = 0$ ✓.

CASE	c_i	$\Delta\Phi_i$	$\hat{c}_i$
no expansion	1	+2	3
expansion ($\text{size}_{i-1} = \text{num}_{i-1} = i - 1$, so $\Phi_{i-1} = i - 1$; after, $\Phi_i = 2$)	i	$2 - (i-1) = 3$ $- i$	3

Both cases give exactly 3. (That the expansion case also lands on 3 is the sanity check that Φ was scaled correctly.)

16.4.2 — Expansion and contraction: the trap

The obvious idea is wrong. "Double when full, halve when less than half full" keeps $\alpha \geq 1/2$, but makes the amortized cost $\Theta(n)$.

The counterexample (this is the shape of counterexample worth remembering): take a table of size $n/2$, n a power of 2. The first $n/2$ operations are insertions, costing $\Theta(n)$, ending with $T.\text{num} = T.\text{size} = n/2$. Then run

```
insert, delete, delete, insert, insert, delete, delete, insert,
insert, ...
```

The first insert expands to size n . The next two deletes contract back to $n/2$. Two more inserts expand again. Each expansion and contraction costs $\Theta(n)$ and there are $\Theta(n)$ of them $\Rightarrow \Theta(n^2)$ total, $\Theta(n)$ amortized per operation.

The diagnosis: after the table expands, not enough deletions occur to pay for a contraction; after it contracts, not enough insertions occur to pay for an expansion. The two thresholds are adjacent, so the structure can be parked on the boundary and thrashed.

The fix: separate the thresholds. Keep doubling on full, but halve only when a deletion drops the table below $1/4$ full. Then:

- the load factor is bounded below by $1/4$;
- immediately after *either* an expansion or a contraction, $\alpha = 1/2$ — the same "home" state;
- from $\alpha = 1/2$ you need $\text{size}/2$ insertions to reach $\alpha = 1$, or $\text{size}/4$ deletions to reach $\alpha = 1/4$. Either way, a constant fraction of the table's worth of operations must occur before the next expensive event.

Verified: driving both strategies to a full table of 4096 and then running the alternating insert, delete, delete, insert sequence for 2000 operations:

STRATEGY	ELEMENTARY MOVES FOR 2000 THRASH OPS	PER OP
halve at $\alpha < 1/2$ (naive)	4 097 500	≈ 2048
halve at $\alpha < 1/4$ (CLRS)	6 096	≈ 3

A **683×** difference on the same operation sequence, from moving one threshold.

The potential function for insert + delete

Set $\Phi = 0$ at $\alpha = 1/2$ (the state after either an expansion or a contraction), and make Φ climb to $T.num$ by the time α reaches either 1 or $1/4$.

- **Above half full** ($\alpha \geq 1/2$): as before, +2 per insertion is needed, so $\Phi(T) = 2(T.num - T.size/2)$.
- **Below half full** ($1/4 \leq \alpha < 1/2$): from $\alpha = 1/2$ it takes $T.size/4$ deletions to reach $\alpha = 1/4$, and the contraction then costs $T.size/4$. So Φ must climb from 0 to $T.size/4$ over $T.size/4$ deletions — **+1 per deletion**. That is produced by $\Phi(T) = T.size/2 - T.num$.

Putting them together:

$$\Phi(T) = \begin{cases} 2(T.num - T.size/2) & \text{if } \alpha(T) \geq 1/2 \\ T.size/2 - T.num & \text{if } \alpha(T) < 1/2 \end{cases}$$

(16.5)

Φ is continuous at $\alpha = 1/2$ (both branches give 0), is 0 for an empty table, and is never negative.

Every case, with its amortized cost:

i-TH OPERATION	c_i	$\Delta\Phi_i$	$\hat{c}_i$
insert, α stays $\geq 1/2$, no expansion	1	+2	3
insert, triggers expansion	i	$3 - i$	3
insert, α stays $< 1/2$	1	-1	0
insert, takes α from below $1/2$ to $= 1/2$	1	-1	0
delete, α stays $\geq 1/2$	1	-2	-1
delete, takes α from $1/2$ to below $1/2$	1	+1	2
delete, α stays $< 1/2$, no contraction	1	+1	2
delete, triggers contraction	$\text{size}_{i-1}/4$	$1 - \text{size}_{i-1}/4$	1

Maximum amortized cost is 3, so any sequence of n operations costs $O(n)$. ■

Worth working through: the contraction row. Before it, $\text{num}_{i-1} = \text{size}_{i-1}/4$, so $\Phi_{i-1} = \text{size}_{i-1}/2 - \text{size}_{i-1}/4 = \text{size}_{i-1}/4$ — which is **exactly the actual cost** of deleting one item and copying the remaining $\text{size}_{i-1}/4 - 1$. After the contraction $\text{num}_i = \text{size}_i/2 - 1$, so $\Phi_i = 1$. The potential built up by the deletions pays the whole contraction, leaving $\hat{c}_i = 1$.

Skiena Problem 3-5 poses exactly this: (a) find a sequence where "halve when below half full" has bad amortized cost — that's the alternating sequence above; (b) give a better underflow strategy achieving constant amortized cost per deletion — that's the $1/4$ threshold.

Outside / Engineering Context

- **Growth factor 2 vs 1.5.** Any constant factor > 1 gives $O(1)$ amortized insert — the analysis only needs a geometric series. Growth factor g costs $g/(g-1)$ moves per element amortized: 2 for $g = 2$, 3 for $g = 1.5$. `libstdc++`'s `std::vector` uses $g = 2$; MSVC uses $g = 1.5$. The argument for 1.5 is allocator-friendliness (with $g = 2$, the sum of all previous blocks is always smaller than the next request, so freed blocks can never be coalesced and reused); the argument for 2 is fewer reallocations. Both are $O(1)$ amortized.
- **`std::vector` never shrinks on its own.** `pop_back` and `clear` don't reduce `capacity()` — precisely to avoid the thrashing above. You must ask, via `shrink_to_fit()` or the swap trick.
- **Hash tables use the same rule.** Rehashing at a load-factor threshold (`max_load_factor`, default 1.0 in `std::unordered_map`) is the dynamic-table analysis with $\Theta(n)$ rehash cost, which is why `unordered_map::insert` is $O(1)$

amortized, not worst-case. Exercise 16.4-2 notes that for an open-addressed table you should call it full at some $\alpha < 1$ (probe counts blow up as $\alpha \rightarrow 1$, per M07), and that the expected actual cost of an individual insertion is still not $O(1)$ — only the amortized-expected cost is.

- **Incremental / de-amortized resizing.** Real-time systems replace the single $\Theta(n)$ rehash with moving $O(1)$ entries per operation while both tables are live, converting an amortized bound into a worst-case one at the cost of complexity. Redis does this.

C++ Implementation — the two running examples and the dynamic table

```
#include <algorithm>
#include <cstdint>
#include <vector>

// ----- stack with
MULTIPOP
template <class T>
class MultipopStack {
public:
    void push(const T& value) { stack_.push_back(value); ++work_; }

    bool pop() {
        if (stack_.empty()) return false;
        stack_.pop_back();
        ++work_;
        return true;
    }

    // Pops min(k, size) elements; actual cost is the number
    popped.
    size_t multipop(size_t k) {
        const size_t popped = min(k, stack_.size());
        stack_.resize(stack_.size() - popped);
        work_ += popped;
        return popped;
    }

    bool empty() const { return stack_.empty(); }
    size_t size() const { return stack_.size(); }
```

```

    size_t work() const { return work_; }           // total elementary
pushes+pops

private:
    vector<T> stack_;
    size_t work_ = 0;
};

// ----- binary
counter
class BinaryCounter {
public:
    explicit BinaryCounter(int k) : bits_(k, 0) {}

    // Returns the number of bits flipped (the actual cost).
    int increment() {
        int i = 0, flips = 0;
        const int k = (int)bits_.size();
        while (i < k && bits_[i] == 1) { bits_[i] = 0; ++i;
++flips; }
        if (i < k) { bits_[i] = 1; ++flips; }
        return flips;
    }

    unsigned long long value() const {
        unsigned long long total = 0;
        for (int i = (int)bits_.size() - 1; i >= 0; --i) total =
total * 2 + bits_[i];
        return total;
    }

    int ones() const { return (int)count(bits_.begin(),
bits_.end(), 1); }

private:
    vector<unsigned char> bits_;
};

// ----- dynamic
table
// Doubles when full; halves when the load factor drops below 1/4.
class DynamicTable {
public:

```

```

void insert(int value) {
    if (size_ == 0) { data_.assign(1, 0); size_ = 1; }
    if (num_ == size_) {
        vector<int> bigger(size_ * 2);
        for (size_t i = 0; i < num_; ++i) { bigger[i] =
data_[i]; ++work_; }
        data_.swap(bigger);
        size_ *= 2;
    }
    data_[num_++] = value;
    ++work_;
}

void remove() {                                     // delete the last
item
    if (num_ == 0) return;
    --num_;
    ++work_;
    if (size_ > 0 && num_ * 4 < size_) {
        const size_t half = size_ / 2;
        vector<int> smaller(half);
        for (size_t i = 0; i < num_; ++i) { smaller[i] =
data_[i]; ++work_; }
        data_.swap(smaller);
        size_ = half;
        if (num_ == 0) { data_.clear(); size_ = 0; }
    }
}

size_t num() const { return num_; }
size_t capacity() const { return size_; }
size_t work() const { return work_; }           // elementary
insertions+deletions

// CLRS potential (16.5): 0 exactly when the load factor is
1/2.
double potential() const {
    if (size_ == 0) return 0.0;
    const double alpha = (double)num_ / (double)size_;
    return alpha >= 0.5 ? 2.0 * ((double)num_ - (double)size_ /
2.0)
                        : (double)size_ / 2.0 - (double)num_;
}

```



```

    }

private:
    vector<int> data_;
    size_t num_ = 0, size_ = 0, work_ = 0;
};

```

Verified: 300 000 random insert/delete operations on `DynamicTable` performed 307 662 elementary moves — an average of **1.03 per operation**, against the proved bound of 3. The potential never went negative (max observed 676), and the load factor never dropped below $1/4$ except when the table was empty.

Part 4 — Applications and Exercises Worth Knowing

Queue from two stacks — amortized $O(1)$ (Exercise 16.3-5)

The structure is in M06; here is its analysis. `ENQUEUE` pushes onto the `in` stack; `DEQUEUE` pops from `out`, first pouring all of `in` into `out` if `out` is empty.

Potential: $\Phi = 2 \cdot |\text{in}|$ (each element in `in` will be moved twice: once out of `in`, once into `out`).

- `ENQUEUE`: $c = 1$, $\Delta\Phi = +2$, $\hat{c} = 3$.
- `DEQUEUE` with `out` nonempty: $c = 1$, $\Delta\Phi = 0$, $\hat{c} = 1$.
- `DEQUEUE` triggering a pour of k elements: $c = 2k + 1$, $\Delta\Phi = -2k$, $\hat{c} = 1$.

Amortized $O(1)$ per operation. **Each element is pushed and popped at most twice in its lifetime** — the same "aggregate" argument as `MULTIPOP`.

Multiset with DELETE-LARGER-HALF (Exercise 16.3-6)

Support `INSERT(S, x)` and `DELETE-LARGER-HALF(S)` (delete the largest $\lceil |S|/2 \rceil$ elements), such that any sequence of m operations runs in $O(m)$.

How: keep an unsorted array. `DELETE-LARGER-HALF` uses linear-time selection (`std::nth_element`, i.e. quickselect from M05) to partition around the median and truncate — actual cost $\Theta(|S|)$.

Potential: $\Phi = 2|S|$.

- INSERT: $c = 1$, $\Delta\Phi = +2$, $\hat{c} = 3$.
- DELETE-LARGER-HALF on a set of size s : $c = \Theta(s)$, and the set shrinks to $\lfloor s/2 \rfloor$, so $\Delta\Phi \approx -s$ and $\hat{c} = O(1)$.

Verified: 200 000 operations (25 091 halvings) cost 512 056 units of work, under the $3m = 600\,000$ bound. Output of all elements is $\Theta(|S|)$ by just reading the array.

```
#include <algorithm>
#include <cstdlib>
#include <vector>

class HalvingMultiset {
public:
    void insert(int value) { elements_.push_back(value); ++work_; }

    void deleteLargerHalf() {
        if (elements_.empty()) return;
        const size_t keep = elements_.size() / 2;           //
        floor(|S|/2) smallest survive
        work_ += elements_.size();
        // nth_element is Theta(|S|)
        if (keep == 0) { elements_.clear(); return; }
        nth_element(elements_.begin(), elements_.begin() +
            (long)keep, elements_.end());
        elements_.resize(keep);
    }

    size_t size() const { return elements_.size(); }
    size_t work() const { return work_; }
    vector<int> elements() const { return elements_; }      //
    Theta(|S|) output

private:
    vector<int> elements_;
    size_t work_ = 0;
};
```

Making binary search dynamic — the logarithmic method (Problem 16-2)

Problem. A sorted array gives $O(\lg n)$ search but $\Theta(n)$ insert. Fix the insert without giving up much search.

Structure. Let $k = \lceil \lg(n+1) \rceil$ and write n in binary as $\langle n_{k-1}, \dots, n_0 \rangle$. Maintain k sorted arrays $A_0, \dots, A_{k-1}$ where A_i has length 2^i and is **full** iff $n_i = 1$, empty otherwise. Each array is individually sorted; elements in different arrays are unrelated.

- **SEARCH:** binary search each nonempty array. Worst case $\sum_i O(\lg 2^i) = O(\lg^2 n)$.
- **INSERT:** make a singleton run and **carry**: while A_i is occupied, merge it with the carry into a run of size 2^{i+1} and continue; deposit the carry in the first empty slot. **This is INCREMENT on a binary counter where flipping bit i from 1 to 0 costs 2^i .** Worst case $\Theta(n)$ (when all bits carry), amortized $\sum_i (n/2^i) \cdot 2^i / n = O(\lg n)$.
- **DELETE:** find the element ($O(\lg^2 n)$), and find any element in the smallest nonempty array A_j ; overwrite the deleted element with it, then re-sort/re-split. Worst case $O(n)$, amortized $O(\lg^2 n)$.

Verified: 60 000 insertions copied 855 072 elements in total — **14.25 per insert**, against $\lg n = 16$. Searches agreed with `std::set` on 4000 present-and-absent probes.

```
#include <algorithm>
#include <cstdint>
#include <iterator>
#include <vector>

class LogMethodSet {
public:
    // Search every nonempty run:  $O(\lg^2 n)$  worst case.
    bool contains(int key) const {
        for (const auto& run : runs_)
            if (!run.empty() && binary_search(run.begin(),
run.end(), key))
                return true;
        return false;
    }

    // Insert a singleton, then carry: merge equal-sized runs,
    exactly like
    // INCREMENT on a binary counter where flipping bit  $i$  costs
     $2^i$ .
```

```

void insert(int key) {
    vector<int> carry{key};
    size_t i = 0;
    while (i < runs_.size() && !runs_[i].empty()) {
        carry = mergeRuns(runs_[i], carry);
        runs_[i].clear();
        runs_[i].shrink_to_fit();
        ++i;
    }
    if (i == runs_.size()) runs_.emplace_back();
    runs_[i] = move(carry);
    ++count_;
}

size_t size() const { return count_; }
size_t work() const { return work_; }      // total elements
copied during merges

vector<int> sorted() const {
    vector<int> merged;
    for (const auto& run : runs_) merged.insert(merged.end(),
run.begin(), run.end());
    sort(merged.begin(), merged.end());
    return merged;
}

private:
    vector<vector<int>> runs_;                // runs_[i] is empty or
has 2^i elements
    size_t count_ = 0;
    mutable size_t work_ = 0;

    // NAME LOOKUP LESSON: this helper is deliberately NOT called
`merge`.
    // A class member name hides every namespace-scope name with
the same
    // identifier, so inside the class a call to `merge(...)` would
find only
    // this member — `using namespace std;` would not rescue it,
and you would
    // get "candidate expects 2 arguments, 5 provided". Rename, or
write

```

```

// `::merge` / `std::merge`. Renaming is the honest fix.
vector<int> mergeRuns(const vector<int>& lhs, const
vector<int>& rhs) {
    vector<int> out;
    out.reserve(lhs.size() + rhs.size());           // one
allocation, not log(n) of them
    // std::merge from <algorithm>: consumes two SORTED ranges
and writes the
    // sorted union to an output iterator. back_inserter turns
push_back into
    // an output iterator, so `out` grows as merge writes.
    merge(lhs.begin(), lhs.end(), rhs.begin(), rhs.end(),
back_inserter(out));
    work_ += out.size();                           // count elements
copied, for the amortized bound
    return out;                                    // return-by-
value: C++11 MOVES this vector out
}
};

```

Outside / Engineering Context

*This is an LSM-tree. LevelDB, RocksDB, Cassandra, HBase and every modern write-optimized store use exactly this shape: sorted runs of geometrically increasing size, merged when two of the same size collide, searched by probing each run (with Bloom filters from M07 to skip runs that certainly don't contain the key — which is what reduces $O(\lg^2 n)$ search to roughly one real lookup). The name in the data-structures literature is the **logarithmic method** or **Bentley–Saxe transformation**: a general recipe for turning any static structure with construction time $C(n)$ and query time $Q(n)$ into a dynamic one with $O((C(n)/n) \cdot \lg n)$ amortized insert and $O(Q(n) \cdot \lg n)$ query.*

Binary reflected Gray code (Problem 16-1)

A Gray code orders the integers so that consecutive values differ in **exactly one bit**. The binary reflected Gray code is built recursively: for $k = 1$ it's $\{0, 1\}$; for $k \geq 2$ take the code for $k-1$, append its reverse with 2^{k-1} added to each element. For $k = 3$: $\{000, 001, 011, 010, 110, 111, 101, 100\}$.

- (a) Which bit flips at step i ? The number of **trailing zeros of i** . (Equivalently, the closed form is $\text{gray}(i) = i \text{ XOR } (i \gg 1)$.)

- **(b) Generating the whole sequence in $\Theta(2^k)$** : start at 0 and flip one bit per step. Total cost is 2^k flips — one per step — rather than $\Theta(k \cdot 2^k)$ to recompute each value.

Note the connection: the ordinary binary counter flips ~ 2 bits per increment amortized; the **Gray counter flips exactly 1, worst case**. That is the de-amortization of the binary counter, and it is why Gray codes are used in rotary encoders and asynchronous clock-domain crossings — no transient state is ever visible mid-flip.

```
#include <vector>

static inline unsigned long long grayCode(unsigned long long i) {
    return i ^ (i >> 1); }

// Which bit flips going from Gray(i-1) to Gray(i): the number of
// trailing zeros of i.
static inline int grayFlippedBit(unsigned long long i) {
    int bitIndex = 0;
    while ((i & 1ULL) == 0) { i >>= 1; ++bitIndex; }
    return bitIndex;
}

// Build the whole sequence in Theta(2^k) by flipping one bit at a
// time.
static vector<unsigned long long> grayCodeSequence(int k) {
    const unsigned long long n = 1ULL << k;
    vector<unsigned long long> out(n);
    unsigned long long current = 0;
    out[0] = 0;
    for (unsigned long long i = 1; i < n; ++i) {
        current ^= (1ULL << grayFlippedBit(i));
        out[i] = current;
    }
    return out;
}
```

Verified for $k = 16$: the 65 536 values form a permutation of $0 \dots 2^{16}-1$, consecutive values differ in exactly one bit, each generated value equals $i \text{ XOR } (i \gg 1)$, and the wraparound from last to first also differs in one bit.

Amortized weight-balanced trees (Problem 16-3)

An alternative to red-black balancing worth knowing because the potential function is instructive. Augment a BST with `x.size`. Call `x` **α -balanced** if `x.left.size` $\leq \alpha \cdot x.size$ and `x.right.size` $\leq \alpha \cdot x.size$, for a constant $1/2 \leq \alpha < 1$; the tree is α -balanced if every node is. Searching an α -balanced tree is $O(\lg n)$ worst case (the subtree size shrinks by a factor α each level, so depth is $\leq \log_{1/\alpha} n$).

The scheme: insert/delete as usual; afterwards, if some node is no longer α -balanced, **rebuild the subtree at the highest such node** into a perfectly $(1/2)$ -balanced tree, in $\Theta(\text{size})$ time.

The potential:

$$\begin{aligned}\Delta(x) &= |x.\text{left.size} - x.\text{right.size}| \\ \Phi(T) &= c \cdot \sum_{\{x \in T : \Delta(x) \geq 2\}} \Delta(x)\end{aligned}$$

Any BST has $\Phi \geq 0$, and a $1/2$ -balanced tree has $\Phi = 0$. Insertions and deletions change $\Delta(x)$ by at most 1 for each of the $O(\lg n)$ nodes on one path, so they add $O(\lg n)$ potential; a rebuild of an m -node subtree that was out of balance has accumulated $\geq m$ units of potential to spend. Amortized cost per insert/delete: $O(\lg n)$.

The same idea, with the rebuild threshold on subtree *counts* rather than heights, is a **scapegoat tree** — no balance bits stored at all.

The cost of restructuring red-black trees (Problem 16-4)

We know `RB-INSERT` and `RB-DELETE` do $O(1)$ rotations — but they can do $\Theta(\lg n)$ **color changes**. Problem 16-4 shows that any sequence of m insertions and deletions on an initially empty red-black tree causes only $O(m)$ **structural modifications total**.

The key observation (and the reason to know this problem): the fixup cases split into **terminating** cases (which end the loop after $O(1)$ more work) and **non-terminating** ones (which iterate). For `RB-INSERT-FIXUP`, case 1 is non-terminating; cases 2 and 3 terminate. For `RB-DELETE-FIXUP`, case 2 is non-terminating; cases 1, 3, 4 terminate. Then:

- **Insertions only:** let $\Phi(T)$ = the number of red nodes. Case 1 of `RB-INSERT-FIXUP` makes two red nodes black and one black node red, so $\Phi(T') = \Phi(T) - 1$ — each non-terminating iteration *pays for itself out of potential*. Amortized structural modifications per `RB-INSERT`: $O(1)$.

- **Insertions and deletions:** use

$$w(x) = \begin{cases} 0 & \text{if } x \text{ is red} \\ 1 & \text{if } x \text{ is black with no red children} \\ 0 & \text{if } x \text{ is black with one red child} \\ 2 & \text{if } x \text{ is black with two red children} \end{cases}$$

$$\Phi(T) = \sum_{x \in T} w(x)$$

Every non-terminating case of *either* fixup satisfies $\Phi(T') \leq \Phi(T) - 1$, so both are $O(1)$ amortized. ■

Why you care: this is what makes the augmentation bound of M08 (Theorem 17.1) tight in practice, and it is the answer to "isn't red-black rebalancing expensive?" — no, in aggregate it is $O(1)$ structural modifications per operation, with only the $O(\lg n)$ search dominating.

Amortized Analysis Elsewhere in the Course

STRUCTURE	BOUND	METHOD	MODULE
Dynamic array / <code>std::vector</code>	$O(1)$ amortized <code>push_back</code>	aggregate or potential	M06, here
Hash table with rehashing	$O(1)$ amortized-expected insert	dynamic-table analysis	M07
Queue from two stacks	$O(1)$ amortized per op	potential $2 \cdot \lceil \ln n \rceil$	M06, here
Red-black restructuring	$O(m)$ modifications for m ops	potential over node colors	M08, here
Disjoint-set forests	$O(m \alpha(n))$ for m ops	potential method (a hard one)	M10
Fibonacci heaps	$O(1)$ amortized <code>INSERT/DECREASE-KEY</code> , $O(\lg n)$ <code>EXTRACT-MIN</code>	potential $t(H) + 2m(H)$	M15 (as the tool behind Dijkstra/Prim)
Splay trees	$O(\lg n)$ amortized per access	potential = sum of log subtree sizes	M08 (named), here (the technique)
Aho–Corasick / KMP failure links	$O(n)$ total scanning	aggregate (the pointer only moves forward)	M18

Note CLRS's own forward reference: "*Aho, Hopcroft, and Ullman used aggregate analysis to determine the running time of operations on a disjoint-set forest. We'll analyze this data structure using the potential method in Chapter 19.*" That is M10, and it is the hardest amortized analysis in the book — "*the amortized analysis that proves this time bound is as complex as the data structure is simple.*"

Also worth knowing from the chapter notes: **potential functions prove lower bounds too**. Define Φ on configurations, compute Φ_{init} , Φ_{final} , and the maximum change $|\Delta\Phi|_{\text{max}}$ any single step can cause; then the number of steps is at least $|\Phi_{\text{final}} - \Phi_{\text{init}}| / |\Delta\Phi|_{\text{max}}$. This is the standard technique for I/O-complexity lower bounds.

Common Mistakes

MISTAKE	WHY IT'S WRONG
Implementing the credit (<code>x.credit</code> fields) in the code	The charges are an analysis device only. CLRS says this explicitly.
Forgetting to check $\Phi \geq 0$ (or $\Phi(D_i) \geq \Phi(D_0)$)	Without it, $\sum \hat{c}$ is not an upper bound on $\sum c$ and the whole argument is void
Letting credit go negative "temporarily, it evens out"	Then the bound fails at that prefix of the sequence; amortized bounds must hold for <i>every</i> prefix
Calling an amortized bound an average-case bound	Amortized involves no probability and holds on adversarial input
Assuming amortized $O(1)$ means no operation ever stalls	A single operation can still be $\Theta(n)$. Fatal in real-time contexts
Setting expansion and contraction thresholds adjacent	The thrashing counterexample: $\Theta(n)$ amortized
Amortizing across independent structures, or reusing a structure after "spending" its potential	Potential belongs to one structure's lifetime; sequences must start from D_0
Ignoring the boundary cases in the case analysis	The dynamic-table analysis has eight cases; four of them are boundary crossings and one of them ($\hat{c} = -1$) looks wrong until you accept that amortized costs may be negative

Recognition Patterns

Reach for amortized analysis when you see:

- **"a single operation might be expensive, but only rarely"** — resizing, rebuilding, rehashing, compaction, garbage collection.
- **A "cleanup" operation whose cost is proportional to the mess made by earlier cheap operations** — `MULTIPOP`, `DELETE-LARGER-HALF`, LSM compaction, path compression.
- **A counter or index structure with cascading carries** — binary counter, logarithmic method, binomial/Fibonacci heaps.
- **A pointer or index that only moves in one direction across a whole run of operations** — KMP's failure-link retreats, two-pointer sliding windows,

monotonic-stack problems (each element pushed and popped once $\Rightarrow O(n)$ total even though a single step pops many).

- **An interview phrase like "on average" applied to a deterministic structure** — that is amortized, and saying so precisely is a strong signal.

In interviews, the monotonic-stack / sliding-window family is where you will actually use this. "Largest rectangle in a histogram", "next greater element", "trapping rain water", "sliding window maximum" all have inner `while` loops that look $O(n)$ per iteration; the correct analysis is the **MULTIPOP** argument — *each element enters and leaves the stack at most once, so the total is $O(n)$* . Say that sentence and the analysis is done.

One-Page Recall

- **Amortized $\neq$ average-case.** No probability; a worst-case guarantee about the whole sequence. Ranking: worst-case $>$ amortized $>$ average-case.
 - **Aggregate:** bound the total $T(n)$, divide. All ops get $T(n)/n$.
 - **Accounting:** charge $\hat{c}_i$, store surplus as **credit on objects**. Require $\sum \hat{c}_i \geq \sum c_i$, i.e. credit never negative. Put the credit on the thing that will later be expensive to destroy.
 - **Potential:** $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$; sum telescopes to $\sum c_i + \Phi(D_n) - \Phi(D_0)$. Require $\Phi(D_i) \geq \Phi(D_0)$; easiest is $\Phi(D_0) = 0, \Phi \geq 0$. Amortized costs may be negative.
 - **Designing Φ :** set $\Phi = 0$ at the state right after the expensive operation; scale so Φ grows to the expensive operation's cost by the time it fires again.
 - **MULTIPOP stack:** $\Phi = |\text{stack}|$. Push 2, pop 0, multipop 0. *An object can't be popped unless it was pushed.*
 - **Binary counter:** $\Phi = \#1\text{-bits}$. $\hat{c} \leq 2$. Bit i flips $n/2^i$ times; $\sum < 2n$. Non-zero start: $\sum c_i \leq 2n - b_n + b_0$.
 - **Dynamic table, insert only:** double on full. $\Phi = 2(\text{num} - \text{size}/2)$. $\hat{c} = 3$ in every case. Skiena: each element moves twice on average.
 - **Dynamic table, insert + delete:** double on full, **halve at $\alpha < 1/4$** (not $1/2$ — that thrashes at $\Theta(n)$ amortized). $\Phi = 2(\text{num} - \text{size}/2)$ if $\alpha \geq 1/2$, else $\text{size}/2 - \text{num}$. Max $\hat{c} = 3$; α stays in $[1/4, 1]$.
 - **The recognition sentence** for the whole module: *"each element is pushed and popped at most once, so the total is $O(n)$."*
-

Practice — where to drill this module

Amortized analysis has few problems *about* it and shows up inside almost every design problem. These are the ones where the amortized argument is the difference between an accepted and a rejected solution.

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
Dynamic array growth, live	380 · Insert Delete GetRandom O(1)	a <code>vector</code> that grows and a swap-with-last erase; the $O(1)$ in the title is amortized , and saying so is part of the answer
Build the table that rehashes	706 · Design HashMap · 705 · Design HashSet	rehashing is <code>TABLE-INSERT</code> under another name — A3
Amortized $O(1)$ per op across a sequence	225 · Implement Stack using Queues	the "expensive push, cheap pop" version is $O(1)$ amortized; explaining <i>why</i> is the interview
Every element enters and leaves once	146 · LRU Cache · 460 · LFU Cache	the aggregate argument of <code>MULTIPOP</code> : total work is bounded by total insertions, not by per-op worst case
The monotonic-stack family	739 · Daily Temperatures · 84 · Largest Rectangle in Histogram	the inner <code>while</code> looks $O(n)$, so the whole thing looks $O(n^2)$ — it is $O(n)$, by exactly the <code>MULTIPOP</code> argument , and this is the single highest-value application of the module
Amortized growth in a stream	703 · Kth Largest Element in a Stream	heap resizing plus a bounded-size invariant

Beyond LeetCode. [CSES Problem Set](#) — *Sorting and Searching* (several are monotonic-stack/two-pointer amortized arguments). [Codeforces](#) `data structures` tag · `two pointers` tag — the two-pointer technique is an amortized argument wearing a different hat: each pointer only moves forward, so the total work is $O(n)$ however the inner loop looks.

The drill that matters here: whenever you write a loop with an inner `while`, ask "can I bound the total number of inner iterations across the whole run?" If yes, you have an amortized bound and the naive $O(n^2)$ is wrong. Monotonic stacks, two pointers, and union-find (M10) are all this one question.

C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., §1.5 and §3.4 (the `vector` implementation).

1. `vector` growth — the module's running example, in the standard library

`std::vector` is TABLE-INSERT. Three members expose the machinery:

```
void growthDemo() {
    vector<int> numbers;
    numbers.size();           // how many elements are in it
    numbers.capacity();       // how many it can hold before the next
                              // reallocation
    numbers.reserve(1000);    // pre-allocate: no reallocation until
                              // size exceeds 1000
    numbers.shrink_to_fit();   // a NON-BINDING request to release
                              // spare capacity
}
```

The standard does not fix the growth factor; it only requires that `push_back` be **amortized $O(1)$** , which forces geometric growth. `libstdc++` and `libc++` double (factor 2); MSVC uses 1.5. **Any constant factor > 1 gives amortized $O(1)$; growing by a constant amount gives $\Theta(n)$ per insert on average** — that is the entire content of §2 of this module, and the standard encodes it as a complexity requirement rather than a policy.

2. Reallocation invalidates everything

```

void danger(vector<int>& numbers) {
    int* first = &numbers[0];
    numbers.push_back(42);    // MAY reallocate...
    // *p is now a dangling pointer if it did. Same for iterators
    and references.
}

```

This is the hidden cost the amortized analysis does not mention: a $\Theta(n)$ copy is *also* a moment when every outstanding pointer into the vector dies. `reserve()` up front removes both the copies and the invalidation. (Contrast `std::list` and `std::deque`, which never invalidate references to existing elements on insertion — see [M06](#).)

3. `noexcept` moves, and why they change the constant

When a `vector` reallocates, it must relocate every element. It **moves** them only if the element's move constructor is `noexcept`; otherwise it **copies**, to preserve the strong exception guarantee. So a type with a non-`noexcept` move turns every doubling from a pointer shuffle into a deep copy. Weiss's Big-Five discussion [§1.5.6, p.30] is the background; the practical rule is one word:

```

struct Buf {
    vector<int> data;
    Buf(Buf&& other) noexcept = default;    // the `noexcept` is
    load-bearing
};

```

4. `mutable`, for instrumentation that does not change logical state

[A3](#) counts elementary moves from inside a `const` accessor. `mutable` is the keyword that permits it:

```

class Counted {
public:
    int value() const { ++reads_; return value_; }    // legal only
    because reads_ is mutable
private:
    int value_ = 0;
    mutable long long reads_ = 0;    // not part of the object's
    logical state
};

```

Use it for caches, counters and locks — never to smuggle real mutation past a `const` promise.

5. Integer types for counters — again

The binary counter in `A2` flips up to 2^n bits over n increments, and the dynamic table in `A3` moves up to 2^n elements. Both are `long long`. At $n = 10^9$ an `int` counter overflows, and signed overflow is **undefined behaviour**, not wraparound.

6. `size_t` arithmetic in a shrinking structure

`A3`'s deletion path compares `num` against `size / 4`. With unsigned types, `num - 1` when `num == 0` wraps to `SIZE_MAX`. Every comparison below is written so the subtraction never happens on an unsigned zero — check emptiness first, or keep the counters signed. This is the same trap as [M02](#) toolkit §2, and it is why the appendix uses `long long` for `num` and `size` rather than `size_t`.

Appendix — C++ for Every Pseudocode Block

A1 MULTIPOP

Pseudocode: §1, "Running example 1 — a stack with MULTIPOP".

```
// A stack with a MULTIPOP that removes min(s, k) items in one
// call.
//
// The elementary-operation counter is what the analysis is about:
// `cost` counts
// individual pushes and pops, NOT calls. A MULTIPOP that pops 1000
// items costs
// 1000, and the whole point is that such a call can only happen
// after 1000
// pushes already paid for those items.
class MultipopStack {
public:
    void push(int value) {
        stack_.push_back(value);
        ++elementaryOps_;           // one elementary operation
    }
};
```

```

    }

    // Returns false on an empty stack rather than throwing: POP on
    empty is a
    // caller error the caller can cheaply avoid, and this keeps
    MULTIPOP's loop
    // simple. (CLRS calls it an error; either convention is
    defensible.)
    bool pop() {
        if (stack_.empty()) return false;
        stack_.pop_back();
        ++elementaryOps_;
        return true;
    }

    // MULTIPOP(S, k)
    void multipop(long long k) {
        // 1  while not STACK-EMPTY(S) and k > 0
        // Note the order: emptiness FIRST. Reversed, the loop
        would still be
        // correct here, but writing the cheap/guarding test first
        is the habit
        // that matters when the second test can fault (M05 A1).
        while (!stack_.empty() && k > 0) {
            stack_.pop_back();          // 2  POP(S)
            ++elementaryOps_;
            k = k - 1;                  // 3  k = k - 1
        }
    }

    bool empty() const { return stack_.empty(); }
    size_t size() const { return stack_.size(); }
    long long elementaryOps() const { return elementaryOps_; }
    void resetCounter() { elementaryOps_ = 0; }
private:
    vector<int> stack_;
    long long elementaryOps_ = 0;    // long long: 2n can exceed int
    (toolkit 5)
};

```


Complexity. A single `MULTIPOP(s, k)` costs $\min(s, k)$, which is $O(n)$ — so the naive bound on n operations is $O(n^2)$. **That bound is correct and useless.**

The aggregate argument. *An object cannot be popped unless it was first pushed.* The number of `POP` calls — including those performed inside `MULTIPOP` — is at most the number of `PUSH` calls, which is at most n . So the total elementary work across any sequence of n operations is at most $2n = O(n)$, and the **amortized cost of each of `PUSH`, `POP` and `MULTIPOP` is $O(1)$.**

The accounting version: charge 2 for each `PUSH` — 1 to perform it, 1 stored as credit *on that object* to pay for its eventual pop. `POP` and `MULTIPOP` then cost 0 amortized, because every pop is paid for by the credit sitting on the item it removes. Credit never goes negative, because you cannot pop an item that was never pushed.

The potential version: $\Phi(D) = |S|$, the number of items on the stack. `PUSH`: $\hat{c} = 1 + 1 = 2$. `POP`: $\hat{c} = 1 - 1 = 0$. `MULTIPOP(k')` where $k' = \min(s, k)$: $\hat{c} = k' - k' = 0$. Same answer, third time.

Where you have already used this: the monotonic stack. In "Daily Temperatures" or "Largest Rectangle in Histogram", the inner `while` pops an unbounded number of elements, so the code *looks* $O(n^2)$ — and is $O(n)$, because each index is pushed once and popped once. **That is `MULTIPOP`, exactly.**

A2 INCREMENT

Pseudocode: §1, "Running example 2 — incrementing a k -bit binary counter".

```
// A k-bit binary counter stored one bit per element, A[0] least
// significant,
// so the value is sum over i of A[i] * 2^i.
//
// vector<char>, NOT vector<bool>: vector<bool> is the bit-packed
// specialisation whose operator[] returns a proxy rather than a
// reference
// (M07 toolkit 7). Here we want ordinary, predictable element
// access.
class BinaryCounter {
public:
    explicit BinaryCounter(int k) : bits_(k, 0) {}

    // INCREMENT(A, k)
```

```

    void increment() {
        int i = 0; // 1 i = 0
        while (i < (int)bits_.size() && bits_[i] == 1) { // 2
while i < k and A[i] == 1
            bits_[i] = 0; // 3
A[i] = 0
            ++flips_; //
(count the bit flip)
            i = i + 1; // 4 i = i
+ 1
        }
        if (i < (int)bits_.size()) { // 5 if i <
k
            bits_[i] = 1; // 6
A[i] = 1
            ++flips_;
        }
        // If the loop ran off the end, the counter OVERFLOWED and
is now all
        // zeros -- CLRS's convention, and the reason line 5 is
guarded.
    }

    long long flips() const { return flips_; } // long long:
the total is ~2n
    int bit(int i) const { return bits_[i]; }
    // The counter's value, for testing. Only valid while it fits
in 64 bits.
    unsigned long long value() const {
        unsigned long long total = 0;
        for (int i = (int)bits_.size() - 1; i >= 0; --i) total =
total * 2 + (unsigned)bits_[i];
        return total;
    }
private:
    vector<char> bits_;
    long long flips_ = 0;
};

```

Complexity. One `INCREMENT` flips up to k bits, so n increments naively cost $O(nk)$.
Correct, not tight.

The aggregate argument. Bit $A[i]$ flips only once every 2^i increments, so over n increments the total is

$$\sum_{i=0}^{k-1} \lfloor n/2^i \rfloor < n \cdot \sum_{i=0}^{\infty} 1/2^i = 2n$$

using the geometric sum $\sum 1/2^i = 2$ from M02. So n increments cost $O(n)$, and the amortized cost per INCREMENT is $O(1)$ — **at most 2 bit flips per increment.**

The potential version, which is the one worth memorizing. Let $\Phi(D_i) = b_i$ = the number of 1-bits after the i -th increment. If the i -th increment resets t_i bits, its actual cost is at most $t_i + 1$, and $b_i \leq b_{i-1} - t_i + 1$. So

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \leq (t_i + 1) + (1 - t_i) = 2$$

The t_i cancels — **that cancellation is the whole trick**, and it is the template for every potential-function argument in the chapter. Starting from zero, $\Phi(D_0) = 0 \leq \Phi(D_n)$, so $\sum \hat{c}_i$ really is an upper bound on $\sum c_i$.

A3 TABLE-INSERT (and TABLE-DELETE)

Pseudocode: §3, "16.4.1 — Expansion only".

```
// A dynamic table that doubles on overflow. The `moves_` counter
// tracks
// ELEMENTARY INSERTIONS -- the quantity the analysis bounds -- so
// the amortized
// claim can be checked rather than believed.
class DynamicTable {
public:
    // TABLE-INSERT(T, x)
    void insert(int value) {
        if (size_ == 0) { // 1 if T.size ==
0
            table_.assign(1, 0); // 2 allocate
1 slot
            size_ = 1; // 3 T.size =
1
        }
        if (num_ == size_) { // 4 if T.num ==
T.size
```

```

        vector<int> fresh(2 * size_);           // 5      allocate
2 * T.size slots
        for (long long i = 0; i < num_; ++i) { // 6      insert
all items into it
            fresh[(size_t)i] = table_[(size_t)i];
            ++moves_;                           //      <-- THE
EXPENSIVE PART:
        }                                       //
Theta(num) elementary moves
        table_.swap(fresh);                   // 7-8      free old,
install new.
        // swap(), not assignment: swapping two vectors
exchanges their
        // internal pointers in O(1) and lets `fresh`'s
destructor free the
        // OLD buffer when it goes out of scope. `table_ =
fresh` would copy.
        size_ = 2 * size_;                     // 9      T.size =
2 * T.size
    }
    table_[(size_t)num_] = value;               // 10 insert x
    ++moves_;
    num_ = num_ + 1;                           // 11 T.num = T.num
+ 1
    }

    // TABLE-DELETE with the QUARTER rule (CLRS 16.4.2).
Contracting at HALF
    // full is the natural-looking choice and it is Theta(n)
amortized: sitting
    // exactly at the boundary, insert-delete-insert-delete forces
a full
    // expansion or contraction on EVERY operation. Waiting until
the table is
    // one QUARTER full leaves a gap between the expansion and
contraction
    // thresholds, so a rebuild is always followed by at least
size/4 cheap
    // operations before the next one. That gap is the whole fix.
    bool erase() {
        if (num_ == 0) return false;           // check emptiness
BEFORE

```

```

        --num_; // decrementing
(toolkit 6)
        ++moves_;
        if (size_ > 1 && num_ > 0 && num_ <= size_ / 4) {
            long long newSize = size_ / 2;
            vector<int> fresh((size_t)newSize);
            for (long long i = 0; i < num_; ++i) {
                fresh[(size_t)i] = table_[(size_t)i];
                ++moves_;
            }
            table_.swap(fresh);
            size_ = newSize;
        }
        return true;
    }

    // The potential function of CLRS (16.5), exposed so it can be
    inspected:
    //      Phi(T) = 2 * T.num - T.size    when the table is at
    least half full
    // It is 0 right after an expansion (num == size/2) and equals
    num right
    // before the next one (num == size), which is exactly enough
    stored credit
    // to pay for copying all num items. THAT is how the function
    is DESIGNED
    // rather than guessed: pick Phi so it is zero after the
    expensive operation
    // and equals the expensive operation's cost just before it.
    long long potential() const { return 2 * num_ - size_; }

    long long num() const { return num_; }
    long long size() const { return size_; }
    long long moves() const { return moves_; }
    void resetCounter() { moves_ = 0; }
private:
    vector<int> table_;
    long long num_ = 0; // items stored
    long long size_ = 0; // slots allocated
    long long moves_ = 0; // elementary insertions performed (long
long: ~2n)
};

```

Complexity.

- A single **TABLE-INSERT** is $\Theta(\text{num})$ in the worst case — the one that triggers a doubling.
- **Amortized $\mathcal{O}(1)$** . Over n insertions starting from an empty table, expansions happen at sizes $1, 2, 4, \dots$, so the total copying work is $1 + 2 + 4 + \dots + 2^{\lceil \lg n \rceil} < 2n$. Adding the n ordinary insertions gives **at most $3n$ elementary operations for n inserts**.
- **Space $\Theta(n)$** , but the table can be up to $2\times$ larger than needed — `shrink_to_fit` exists for exactly this.

Why geometric growth and not a fixed increment. Growing by a constant c slots means expansions at $c, 2c, 3c, \dots$, and the total copying is $c + 2c + \dots + n = \Theta(n^2/c) = \Theta(n^2)$ — **$\Theta(n)$ amortized per insert**. Doubling turns an arithmetic series into a geometric one, and geometric series are dominated by their last term (M02). Any factor > 1 works; 2 and 1.5 are both in production use.

Skiena's warning about what the amortized bound costs you:

"The primary thing lost in using dynamic arrays is the guarantee that each insertion takes constant time in the worst case... What we get instead is a promise that the n th element insertion will be completed quickly enough that the total effort expended so far will still be $\mathcal{O}(n)$."

For a game loop, an audio callback, or anything with a tail-latency budget, that occasional $\Theta(n)$ stall is exactly what you cannot afford. The fixes are to `reserve()` up front, or to use a structure that spreads the work out (incremental rehashing, or the logarithmic method of §5).

Next: [M10 — Disjoint Sets / Union-Find](#) (CLRS 19 + Skiena 8.1.3) — union by rank, path compression, and the $\alpha(n)$ bound: the simplest data structure with the hardest analysis.